

Ingeniería del Software de Componentes y Sistemas Distribuidos

PRUEBA DE EVALUACIÓN CONTINUADA – Práctica 3

Presentación

En la Práctica 2 se han implementado tres microservicios del caso **Photo&Film4You**. En esta Práctica 3 probaremos (*testing*) uno de los tres microservicios y validaremos que nuestro código sigue una arquitectura hexagonal.

La Práctica 3 abarca los contenidos de los libros "Microservices Patterns" y "Fundamentals of Software Architecture" (ver en el apartado Recursos los capítulos que hay que estudiar).

Competencias

En esta Práctica se trabaja las siguientes competencias del Grado de Ingeniería Informática:

- Saber proponer y evaluar diferentes alternativas tecnológicas para resolver un problema concreto.
- Aplicación de las técnicas específicas de la Ingeniería del Software a las diferentes etapas del ciclo de vida de un proyecto.

Objetivos

Los objetivos de la Práctica 3 son:

- Reconocer los factores de testing y calidad de una especificación/diseño.
- Evaluar formalmente el testing y la calidad de una implementación.
- Identificar los modelos más importantes y estándares de calidad de software.

Descripción de la Práctica 3 a realizar

La Práctica 3 se compone de 3 ejercicios. En cada uno de ellos hay que describir **cómo se ha hecho el ejercicio** y aportar **instrucciones** para su correcta ejecución.

Ejercicio 1 (5 puntos)

Partiendo de la solución de la Práctica 2 publicada en: <https://github.com/orgs/UOC-ISCSD-SPRING-2024/repositories>, se deben implementar **pruebas unitarias**, usando **Junit**, **Spring Boot Test**, **Mockito** y **AssertJ**.

Notas: Podéis usar como referencia para ayudaros a desarrollar los tests:

- <https://github.com/anirban99/hexagonal-architecture/tree/master/src/test/java/com/example/hexagonal-architecture>
- <https://github.com/eugenp/tutorials/blob/master/spring-boot-modules/spring-boot-testing/src/test/java/com/baeldung/boot/testing/EmployeeServiceImplIntegrationTest.java>
- <https://spring.io/guides/gs/testing-web/>
- <https://docs.spring.io/spring-boot/docs/current/reference/html/features.html#features.testing.spring-boot-applications.with-mock-environment>

Consejo: En las pruebas unitarias no hacemos uso de ninguna infraestructura ni dependencia real. No usamos comunicación HTTP, servidores, bases de datos, servicios externos u otros microservicios. Todo aspecto relacionado con dependencias debe simularse (con **Mockito**), excepto las clases de dominio (Product, ...).

QUÉ HAY QUE HACER

Desarrollar las siguientes clases y tests:

1. **OfferUnitTest**. Incluir un test sobre la clase de dominio *Offer* que verifique que al crear una *Offer*, su *status* sea PENDING.
2. **OfferServiceUnitTest**. Incluir tests que verifiquen:
 - Método Test 1: Cuando llamamos al método *findOffersByUser* con un *id* existente/válido, obtenemos correctamente el listado de ofertas.
 - Método Test 2: Cuando llamamos al método *findOffersByUser* con un *id* inexistente, obtenemos la excepción que indica que no se ha encontrado el usuario.

Notas:

- La dependencia con el *UserRepository* debe ser simulada.
- La relación de Offer con el usuario se hace a través del campo *email* y no del ID.

3. **OfferControllerUnitTest.** Incluir un test que verifique que si llamamos a */offers?email={value}* obtenemos correctamente las ofertas creadas por el usuario con el correo especificado.

Notas:

- No se puede levantar el servidor para hacer la prueba. Se debe simular la infraestructura web para recibir y tratar la petición REST, así como la dependencia con *OfferService*.

Solución

1.1:

OfferUnitTest

```
package edu.uoc.epcsd.productcatalog;

import org.junit.jupiter.api.Test;

import edu.uoc.epcsd.productcatalog.domain.Offer;
import edu.uoc.epcsd.productcatalog.domain.OfferStatus;

import static org.assertj.core.api.Assertions.assertThat;

public class OfferUnitTest {
    @Test
    public void whenCreatingOffer_thenStatusShouldBePending() {
        // Given
        Long id = 1L;
        String email = "test@example.com";
        Long categoryId = 2L;
        Long productId = 3L;
        String serialNumber = "123456789";

        // When
        Offer offer = Offer.builder()
            .id(id)
            .email(email)
            .categoryId(categoryId)
            .productId(productId)
            .serialNumber(serialNumber)
            .build();

        // Then
        assertThat(offer.getStatus()).isEqualTo(OfferStatus.PENDING);
    }
}
```

1.2:

En este ejercicio hay dos opciones, cualquiera de ellas es válida y cumple la función requerida en el enunciado: usar la extensión de Spring o usar la extensión de Mockito que no requiere Spring

Ejemplo con extensión Spring:

OfferServiceUnitTest

```
package edu.uoc.epcsd.productcatalog;

import edu.uoc.epcsd.productcatalog.application.rest.response.GetUserResponse;
import edu.uoc.epcsd.productcatalog.domain.Offer;
import edu.uoc.epcsd.productcatalog.domain.OfferStatus;
import edu.uoc.epcsd.productcatalog.domain.repository.OfferRepository;
import edu.uoc.epcsd.productcatalog.domain.service.OfferServiceImpl;
import edu.uoc.epcsd.productcatalog.domain.service.UserNotFoundException;

import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.InjectMocks;
import org.mockito.Mock;
import org.mockito.MockitoAnnotations;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.client.HttpClientErrorException;
import org.springframework.web.client.RestTemplate;

import java.util.Arrays;
import java.util.List;

import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertThrows;
import static org.mockito.Mockito.*;

public class OfferServiceUnitTest {

    @InjectMocks
    private OfferServiceImpl offerService;

    @Mock
    private OfferRepository offerRepository;

    @Mock
    private RestTemplate restTemplate;

    @BeforeEach
    public void setUp() {
        MockitoAnnotations.openMocks(this);
    }

    @Test
    public void whenFindOffersByUser_withValidEmail_thenReturnOffersList() {
        // Given
        String email = "valid@example.com";
        Offer offer1 = new Offer(1L, email, 2L, 3L, "123456789", OfferStatus.PENDING, null);
        Offer offer2 = new Offer(2L, email, 2L, 3L, "987654321", OfferStatus.PENDING, null);
        List<Offer> expectedOffers = Arrays.asList(offer1, offer2);

        GetUserResponse getUserResponse = new GetUserResponse();
        ResponseEntity<GetUserResponse> responseEntity = new ResponseEntity<>(getUserResponse, HttpStatus.OK);

        when(restTemplate.getForEntity(any(), eq(GetUserResponse.class), eq(email)))
            .thenReturn(responseEntity);
    }
}
```

```

        when(offerRepository.findOffersByUser(email)).thenReturn(expectedOffers);

        // When
        List<Offer> actualOffers = offerService.findOffersByUser(email);

        // Then
        assertEquals(expectedOffers, actualOffers);
        verify(restTemplate, times(1)).getForEntity(any(), eq(GetUserResponse.class), eq(email));
        verify(offerRepository, times(1)).findOffersByUser(email);
    }

    @Test
    public void whenFindOffersByUser_withInvalidEmail_thenThrowUserNotFoundException() {
        // Given
        String email = "invalid@example.com";

        when(restTemplate.getForEntity(any(), eq(GetUserResponse.class), eq(email)))
            .thenReturn(new HttpClientErrorException(HttpStatus.NOT_FOUND));

        // When / Then
        assertThrows(UserNotFoundException.class, () -> offerService.findOffersByUser(email));
        verify(restTemplate, times(1)).getForEntity(any(), eq(GetUserResponse.class), eq(email));
        verify(offerRepository, times(0)).findOffersByUser(email);
    }
}

```

1.3:

Añadimos configuración para testear sólo los controladores indicados en el enunciado:

OfferControllerUnitTest

```

package edu.uoc.epcsd.productcatalog;

import edu.uoc.epcsd.productcatalog.application.rest.OfferRestController;
import edu.uoc.epcsd.productcatalog.domain.Offer;
import edu.uoc.epcsd.productcatalog.domain.service.OfferService;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import org.mockito.MockitoAnnotations;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.AutoConfigureDataJpa;
import org.springframework.boot.test.autoconfigure.web.servlet.WebMvcTest;
import org.springframework.boot.test.mock.mockito.MockBean;
import org.springframework.http.MediaType;
import org.springframework.test.web.servlet.MockMvc;

import java.util.Arrays;
import java.util.List;

import static org.mockito.Mockito.when;
import static org.springframework.test.web.servlet.request.MockMvcRequestBuilders.get;
import static org.springframework.test.web.servlet.result.MockMvcResultMatchers.*;

@WebMvcTest(OfferRestController.class)
@AutoConfigureDataJpa
public class OfferControllerUnitTest {

    @Autowired
    private MockMvc mockMvc;

    @MockBean
    private OfferService offerService;

    @BeforeEach
    public void setUp() {

```

```
MockitoAnnotations.openMocks(this);
}

@Test
public void whenFindOffersByUser_withValidEmail_thenReturnOffersList() throws Exception {
    // Given
    String email = "valid@example.com";
    Offer offer1 =
Offer.builder().id(1L).email(email).categoryId(2L).productId(3L).serialNumber("123456789").build();
    Offer offer2 =
Offer.builder().id(2L).email(email).categoryId(2L).productId(3L).serialNumber("987654321").build();

    List<Offer> expectedOffers = Arrays.asList(offer1, offer2);

    when(offerService.findOffersByUser(email)).thenReturn(expectedOffers);

    // When & Then
    mockMvc.perform(get("/offers")
        .param("email", email)
        .contentType(MediaType.APPLICATION_JSON))
        .andExpect(status().isOk())
        // Offer1
        .andExpect(jsonPath("$.id").value(offer1.getId()))
        .andExpect(jsonPath("$.email").value(offer1.getEmail()))
        .andExpect(jsonPath("$.categoryId").value(offer1.getCategoryId()))
        .andExpect(jsonPath("$.productId").value(offer1.getProductId()))
        .andExpect(jsonPath("$.serialNumber").value(offer1.getSerialNumber()))
        // Offer2
        .andExpect(jsonPath("$.id").value(offer2.getId()))
        .andExpect(jsonPath("$.email").value(offer2.getEmail()))
        .andExpect(jsonPath("$.categoryId").value(offer2.getCategoryId()))
        .andExpect(jsonPath("$.productId").value(offer2.getProductId()))
        .andExpect(jsonPath("$.serialNumber").value(offer2.getSerialNumber()));
}
```

Ejecución de los Test Runs sobre la solución de la PRAC2

Todos los tests requeridos superan la validación:

Finished after 6,544 seconds

Runs:	Errors:	Failures:
4/4	0	0

- OfferServiceUnitTest [Runner: JUnit 5] (1,169 s)
 - whenFindOffersByUser_withInvalidEmail_thenThrowUserNotFoundException() (1,133 s)
 - whenFindOffersByUser_withValidEmail_thenReturnOffersList() (0,035 s)
- OfferControllerUnitTest [Runner: JUnit 5] (0,249 s)
 - whenFindOffersByUser_withValidEmail_thenReturnOffersList() (0,249 s)
- OfferUnitTest [Runner: JUnit 5] (0,085 s)
 - whenCreatingOffer_thenStatusShouldBePending() (0,085 s)

Ejercicio 2 (3 puntos)

Partiendo de la solución de la PRAC2 publicada en: (<https://github.com/orgs/UOC-ISCSD-SPRING-2024/repositories>), debéis desarrollar **pruebas de integración**, usando **JUnit**, **Spring Boot Test** y **AssertJ**.

Notas:

- Podéis usar como referencia para ayudaros a desarrollar los tests: <https://github.com/eugenp/tutorials/tree/master/spring-boot-modules/spring-boot-testing/src/test/java/com/baeldung/boot/testing>

Consejo: Las pruebas de integración deben probar que el servicio se comunica con sus dependencias reales y la infraestructura. Por lo tanto, no se hace uso de la librería **Mockito** como en las pruebas unitarias.

QUÉ HAY QUE HACER

Desarrollar la siguiente clase con su test:

- **OfferRepositoryIntegrationTest**. Probar que al añadir una Offer podemos encontrarla correctamente con el método *findOffersByUser*.

Solución

Añadimos configuración para testear sólo los repositorios necesarios indicados en el enunciado

OfferRepositoryIntegrationTestConfig

```
package edu.uoc.epcsd.productcatalog;

import edu.uoc.epcsd.productcatalog.infrastructure.repository.jpa.CategoryRepositoryImpl;
import edu.uoc.epcsd.productcatalog.infrastructure.repository.jpa.ProductRepositoryImpl;
import org.springframework.boot.autoconfigure.EnableAutoConfiguration;
import org.springframework.context.annotation.ComponentScan;
import org.springframework.context.annotation.Configuration;
import org.springframework.context.annotation.FilterType;

@Configuration
@EnableAutoConfiguration
@ComponentScan(basePackages =
"edu.uoc.epcsd.productcatalog.infrastructure.repository.jpa", excludeFilters =
@ComponentScan.Filter(type = FilterType.ASSIGNABLE_TYPE, classes =
{CategoryRepositoryImpl.class, ProductRepositoryImpl.class}))
public class OfferRepositoryIntegrationTestConfig {
}
```

OfferRepositoryIntegrationTest

```
package edu.uoc.epcsd.productcatalog;

import edu.uoc.epcsd.productcatalog.domain.Offer;
import edu.uoc.epcsd.productcatalog.domain.repository.OfferRepository;
import edu.uoc.epcsd.productcatalog.domain.OfferStatus;

import org.junit.jupiter.api.Test;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.jdbc.AutoConfigureTestDatabase;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.test.context.ContextConfiguration;
import java.time.LocalDate;

import static org.assertj.core.api.AssertionsForClassTypes.assertThat;

@DataJpaTest
@AutoConfigureTestDatabase(replace = AutoConfigureTestDatabase.Replace.NONE)
@ContextConfiguration(classes = OfferRepositoryIntegrationTestConfig.class)
public class OfferRepositoryIntegrationTest {

    private final String OFFER_EMAIL = "juanpa@gmail.com";
    private final String SERIAL_NUMBER = "TestNumber 978-11111";
    private final Long PRODUCT = 1L;
    private final Long CATEGORY = 1L;
    private final OfferStatus STATUS = OfferStatus.ACCEPTED;
    private final LocalDate DATE = LocalDate.now();

    @Autowired
    private OfferRepository offerRepository;

    @Test
    void whenFindById_thenReturnOffer() {
        Offer offer = Offer.builder().email(OFFER_EMAIL).categoryId(CATEGORY).productId(PRODUCT).serialNumber(SERIAL_NUMBER).status(STATUS).date(DATE).build();
        offerRepository.addOffer(offer);
        Offer fromDb = offerRepository.findOffersByUser(OFFER_EMAIL).get(0);
        assertThat(fromDb.equals(offer));
    }
}
```

Ejecución de los Test Runs sobre la solución de la PRAC2

El test supera la validación:

Finished after 8,229 seconds

Runs: 1/1

✖ Errors: 0

✖ Failures: 0

▼  OfferRepositoryIntegrationTest [Runner: JUnit 5] (0,687 s)
  whenFindByld_thenReturnOffer() (0,687 s)

Ejercicio 3 (2 puntos)

Partiendo de la solución de la PRAC2 publicada <https://github.com/orgs/UOC-ISCS-SPRING-2024/repositories>, queremos verificar que la arquitectura se basa correctamente en una arquitectura hexagonal y que usamos buenas reglas arquitectónicas.

Notas:

- Para hacer el test usaremos **ArchUnit** (<https://www.archunit.org/>) con **Junit5**. Usad la versión 1.0.1 de **ArchUnit**.

Lecturas recomendadas:

- Capítulo "Measuring and Governing Architecture Characteristics" del libro "Fundamentals of Software Architecture"
- Documentación ArchUnit:
https://www.archunit.org/userguide/html/000_Index.html#_junit_4_5_support
- Ejemplos ArchUnit con JUnit5:
<https://github.com/TNG/ArchUnit-Examples/tree/main/example-junit5/src/test>

QUÉ HAY QUE HACER

Desarrollar los siguientes 2 tests automáticos de verificación de la arquitectura sobre el microservicio *UserService*:

- Se cumple la arquitectura hexagonal (llamada onion a **ArchUnit**)
- Las clases que se encuentran en el *package domain.service* y están anotadas con **@Service**, tienen su nombre terminado en **ServiceImpl**.

Solución

Hay que añadir las dependencias de ArchUnit al pom.xml:

```
<dependency>
  <groupId>com.tngtech.archunit</groupId>
  <artifactId>archunit-junit5</artifactId>
  <version>1.0.1</version>
  <scope>test</scope>
</dependency>
```

ArchitectureTest:

```
package edu.uoc.epcsd.user;

import com.tngtech.archunit.core.importer.ImportOption;
import com.tngtech.archunit.junit.AnalyzeClasses;
import com.tngtech.archunit.junit.ArchTest;
import com.tngtech.archunit.lang.ArchRule;
import org.springframework.stereotype.Service;

import static com.tngtech.archunit.lang.syntax.ArchRuleDefinition.classes;
import static com.tngtech.archunit.library.Architectures.onionArchitecture;

@AnalyzeClasses(packages = "edu.uoc.epcsd.user", importOptions =
{ImportOption.DoNotIncludeTests.class, ImportOption.DoNotIncludeJars.class})
public class ArchitectureTest {

    @ArchTest
    static final ArchRule domain_service_with_spring_annotation = classes()
        .that().resideInAPackage("..domain.service..")
        .and().areAnnotatedWith(Service.class)
        .should().haveSimpleNameEndingWith("ServiceImpl");

    @ArchTest
    static final ArchRule onion_architecture_is_respected = onionArchitecture()
        .domainModels("..domain..")
        .domainServices("..domain.service..")
        .applicationServices("..application..")
        .adapter("persistence", "..infrastructure.repository..")
        .adapter("rest", "..application.rest..");
}
```

Ejecución de los Test Runs sobre la solución de la PRAC2

En este caso, el primer test, referente a la validación de la arquitectura hexagonal, no supera la validación. Tal como se ha indicado en el aula durante la realización de la PRAC3, se detecta una serie de dependencias entre clases que no cumplen las reglas definidas.

El segundo test, referente a la validación de la nomenclatura de clases con anotación `@Service`, sí se supera.

```

Finished after 1,667 seconds
Runs: 2/2      Errors: 0      Failures: 1
ArchitectureTestUser [Runner: JUnit 5] (1,49s)
  domain_service_with_spring_annotation
  onion_architecture_is_respected (0,226 s)
    java.lang.AssertionError: Architecture Violation [Priority: MEDIUM] - Rule 'Onion architecture consisting of
    domain models ('..domain..')
    domain services ('..domain.service..')
    application services ('..application..')
    adapter 'persistence' ('..infrastructure.repository..')
    adapter 'rest' ('..application.rest..') was violated (2 times):
    Method <edu.uoc.epcsd.user.domain.service.AlertServiceImpl.createAlert(edu.uoc.epcsd.user.domain.Alert)> references class object
    <edu.uoc.epcsd.user.application.rest.response.GetProductResponse> in (AlertServiceImpl.java:49)
    Method <edu.uoc.epcsd.user.domain.service.AlertServiceImpl.createAlert(edu.uoc.epcsd.user.domain.Alert)> references class object
    <edu.uoc.epcsd.user.application.rest.response.GetProductResponse> in (AlertServiceImpl.java:49)
  
```

Recursos

Recursos Básicos

- Libro "Microservices Patterns" (Chris Richardson): Chapter 9. "Testing microservices: Parte 1". Introduction.
- Libro "Microservices Patterns" (Chris Richardson): Chapter 10. "Testing microservices: Parte 2".
- Libro "Fundamentals of Software Architecture" (Richards & Ford).
 - o Chapter 3. "Modularity".
 - o Chapter 6. "Measuring and Governing Architecture Characteristics".

Criterios de evaluación

- En esta actividad **no está permitido el uso de herramientas de inteligencia artificial** y se tiene que resolver de forma **estrictamente individual**. En caso de detectar irregularidades se penalizará la actividad con una D como nota. Esto incluye la reutilización de soluciones de esta práctica de semestres anteriores. En el plan docente y en el [web sobre integridad académica y plagio de la UOC](#) encontraréis información sobre qué se considera conducta irregular en la evaluación y las consecuencias que puede tener.
- El peso de cada ejercicio está indicado en el enunciado.
- Los criterios de evaluación de cada ejercicio están indicados en el anexo.
- Es necesario justificar la solución a cada uno de los ejercicios. Se valorará tanto la corrección de la solución como la justificación dada.

Formato y fecha de entrega

Hay que entregar un único **documento** PDF *ApellidosNombre_ISCSDPRAC3.pdf* con la descripción de cómo se ha realizado la práctica (argumentar decisiones e instrucciones de cómo se deben ejecutar las pruebas desarrolladas) y entregar el **código de los test desarrollados** (en un fichero ZIP).

El documento PDF y el fichero ZIP se entregarán en el espacio “Entrega de la PRAC3” del aula antes de las **23:59 horas del día 17 de junio de 2024**. No se aceptarán entregas fuera de plazo.

Anexo

Criterios de evaluación de la Práctica 3

	Excelente (10 puntos)	Notable (7 puntos)	Suficiente (5 puntos)	Insuficiente (2,5 puntos)	Sin respuesta / Todo mal (0 puntos)
Ej. 1	• 4 tests implementados y justificados correctamente en los ejercicios 1.1, 1.2 y 1.3. • Uso correcto de Junit en los 4 tests de los ejercicios 1.1, 1.2 y 1.3. • Uso correcto de Mockito en los 3 tests de los ejercicios 1.2 y 1.3. • Separar en 2 métodos los dos tests del ejercicio 1.2. • Calidad general de la implementación perfecta (criterios: legibilidad, formato, sin	• 3 tests implementados y justificados correctamente en los ejercicios 1.1, 1.2 y 1.3. • Uso correcto de Junit en los 4 tests de los ejercicios 1.1, 1.2 y 1.3. • Uso correcto de Mockito en 2 de los 3 tests de los ejercicios 1.2 y 1.3. • Separar en 2 métodos los dos tests del ejercicio 1.2. • Calidad general de la implementación buena (criterios: legibilidad, formato, sin advertencias,	• 2 tests implementados y justificados correctamente en los ejercicios 1.1, 1.2 y 1.3. • Uso correcto de Junit en 2 de los 4 tests de los ejercicios 1.1, 1.2 y 1.3. • Uso correcto de Mockito en 1 de los 3 tests de los ejercicios 1.2 y 1.3. • No separar en 2 métodos los dos tests del ejercicio 1.2. • Calidad general de la implementación suficiente (criterios: legibilidad, formato, sin	• 1 test implementado y justificado correctamente en los ejercicios 1.1, 1.2 y 1.3. • Uso incorrecto de Junit en los 4 tests de los ejercicios 1.1, 1.2 y 1.3. • No se utilizó Mockito en los 3 tests de los ejercicios 1.2 y 1.3. • No se separaron en 2 métodos los dos tests del ejercicio 1.2. • Calidad general de la implementación insuficiente (criterios: legibilidad,	Sin respuesta, no hay ninguna justificación o todo incorrecto (0 puntos)

	advertencias, uso de constantes en lugar de codificación directa en las llamadas a los métodos, etc.). (5 puntos)	uso de constantes en lugar de codificación directa en las llamadas a los métodos, etc.). (3,5 puntos).	advertencias, uso de constantes en lugar de codificación directa en las llamadas a los métodos, etc.). (2,5 puntos).	formato, sin advertencias, uso de constantes en lugar de codificación directa en las llamadas a los métodos, etc.). (1 punto)	
Ej. 2	• Test implementado y justificado correctamente. • Calidad general de la implementación <u>perfecta</u> (criterios: legibilidad, formato, sin advertencias, uso de constantes en lugar de codificación directa en las llamadas a los métodos, etc.). (3 puntos)	• Test implementado correctamente, pero justificado incorrectamente. • Calidad general de la implementación <u>buen</u>a (criterios: legibilidad, formato, sin advertencias, uso de constantes en lugar de codificación directa en las llamadas a los métodos, etc.). (2 puntos)	• Test implementado correctamente, pero sin justificar. • Calidad general de la implementación <u>suficiente</u> (criterios: legibilidad, formato, sin advertencias, uso de constantes en lugar de codificación directa en las llamadas a los métodos, etc.). (1,5 puntos)	• Test con justificación, pero implementado incorrectamente • Calidad general de la implementación <u>insuficiente</u> (criterios: legibilidad, formato, sin advertencias, uso de constantes en lugar de codificación directa en las llamadas a los métodos, etc.). (1 punto)	Sin respuesta, no hay ninguna justificación o todo incorrecto (0 puntos)
Ej. 3	• 2 tests implementados y justificados correctamente. • Calidad general de la implementación <u>perfecta</u> (criterios: legibilidad, formato, sin advertencias, uso de constantes en lugar de codificación directa en las llamadas a los métodos, etc.). (2 puntos)	• 2 tests implementados pero justificados incorrectamente. • Calidad general de la implementación <u>buen</u>a (criterios: legibilidad, formato, sin advertencias, uso de constantes en lugar de codificación directa en las llamadas a los métodos, etc.). (1,5 puntos)	• 1 test implementado y justificado correctamente. • Calidad general de la implementación <u>suficiente</u> (criterios: legibilidad, formato, sin advertencias, uso de constantes en lugar de codificación directa en las llamadas a los métodos, etc.). (1 punto)	• Tests mal implementados y no justificados. • Calidad general de la implementación <u>insuficiente</u> (criterios: legibilidad, formato, sin advertencias, uso de constantes en lugar de codificación directa en las llamadas a los métodos, etc.). (0,5 puntos)	Sin respuesta, no hay ninguna justificación o todo incorrecto (0 puntos)

